Practical No. 7 :-

A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint). The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. *(Sample data for reference is given below).*

Process	Allocation				MAX			
	Printer	ROM	Hard Disk	RAM	Printer	ROM	Hard Disk	RAM
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	0	1	2
Excel	1	0	0	0	1	7	5	0
PowerPoint	1	3	5	4	2	3	5	6

Available Resources			
Printer	ROM	Hard Disk	RAM
1	6	2	0

Write a program to :

- Calculate current need of resources by each process.
- Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

Code :

```
bhavi@LAPTOP-IH23AUG3 MSYS
$ nano program.c
```

```
 GNU nano 8.7                                                    program.c
//C program for Banker's Algorithm
#include <stdio.h>
int main()
{
        // P0, P1, P2, P3, P4 are the names of Process

        int n, r, i, j, k;
        n = 5; // Indicates the Number of processes
        r = 3; //Indicates the Number of resources
        int alloc[5][3] = { { 0, 0, 1 }, // P0 // This is Allocation Matrix
                            { 3, 0, 0 }, // P1
                            { 1, 0, 1 }, // P2
                            { 2, 3, 2 }, // P3
                            { 0, 0, 3 } }; // P4

        int max[5][3] = { { 7, 6, 3 }, // P0 // MAX Matrix
                          { 3, 2, 2 }, // P1
                          { 8, 0, 2 }, // P2
                          { 2, 1, 2 }, // P3
                          { 5, 2, 3 } }; // P4

        int avail[3] = { 2, 3, 2 }; // These are Available Resources

        int f[n], ans[n], ind = 0;
        for (k = 0; k < n; k++) {
                f[k] = 0;
        }
        int need[n][r];
        for (i = 0; i < n; i++) {
                for (j = 0; j < r; j++)
                        need[i][j] = max[i][j] - alloc[i][j];
        }
        int y = 0;
        for (k = 0; k < 5; k++) {
                for (i = 0; i < n; i++) {
                        if (f[i] == 0) {

                                int flag = 0;
                                for (j = 0; j < r; j++) {
                                        if (need[i][j] > avail[j]){
                                                flag = 1;
                                                break;
                                        }
                                }

                                if (flag == 0) {
                                        ans[ind++] = i;
                                        for (y = 0; y < r; y++)
                                                avail[y] += alloc[i][y];
                                        f[i] = 1;
                                }
                        }
                }
        }
}
                                [ Read 63 lines (converted from DOS format) ]
^G Help       ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark  M-] To Bracket  M-B Previous  ^← Back       ^Y Prev Word
^X Exit       ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy      ^Q Where Was    M-F Next      ^→ Forward    ^V Next Word
```

Output :

```
bhavi@LAPTOP-IH23AUG3 MSYS ~
$ ./program
The SAFE Sequence is as follows
p1 -> p3 -> p4 -> p0 -> p2
```